

```
facade-service > main.py > get_user
13 async def handle_post(data: dict = Body(...)):
15     payload = {**data, "transaction_id": t_id}
16

PROBLEMS 46 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User\Documents\3 course\sem2\apz\hw\hw1> docker compose up --build
=> [facade-service] resolving provenance for metadata file
[+] Running 3/3
  ✓ Container hw1-facade-service-1   Recreated
  ✓ Container hw1-counter-service-1  Recreated
  ✓ Container hw1-logging-service-1  Recreated
Attaching to counter-service-1, facade-service-1, logging-service-1
counter-service-1 | INFO: Started server process [1]
counter-service-1 | INFO: Waiting for application startup.
logging-service-1 | INFO: Started server process [1]
counter-service-1 | INFO: Application startup complete.
facade-service-1 | INFO: Started server process [1]
logging-service-1 | INFO: Waiting for application startup.
counter-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
facade-service-1 | INFO: Waiting for application startup.
logging-service-1 | INFO: Application startup complete.
facade-service-1 | INFO: Application startup complete.
logging-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
facade-service-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
logging-service-1 | Logged: 3a59b79a-d8da-477a-9c5f-f5b1d3bde1d9
172.18.0.4:43702 - "POST /log HTTP/1.1" 200 OK
counter-service-1 | INFO: 172.18.0.4:43978 - "POST /transaction HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:58252 - "POST /process HTTP/1.1" 200 OK
facade-service-1 | INFO: 172.18.0.1:58000 - "GET /process HTTP/1.1" 405 Method Not Allowed
```

На скріншотах зафіксовано успішну роботу системи, де facade-service прийняв POST-запит і переслав його на інші мікросервіси. Logging-service зберіг транзакцію в пам'яті та вивів її ID у консоль для відлагодження, а counter-service оновив баланс рахунку. Система також коректно обробила помилку 405, заборонивши GET-запит на адресу, призначену тільки для POST-транзакцій.

Система успішно обробляє базові транзакції. На скріншотах нижче видно:

- POST /process: Створення транзакцій для різних користувачів (user_1, user_2) з позитивними та негативними сумами.
- GET /accounts: Отримання актуальних балансів усіх клієнтів.
- Консоль: logging-service виводить UUID транзакцій у консоль, що підтверджує отримання та збереження даних.

POST method

```

and
PS C:\Users\user> Invoke-RestMethod -Uri "http://localhost:8080/process" -Method Post -Body '{"user_id": "user_1", "amount": 100}' -ContentType "application/json"

transaction_id      balance
-----
b2f09645-b6b7-43d2-854d-b9763d6949ae      100

PS C:\Users\user> Invoke-RestMethod -Uri "http://localhost:8080/process" -Method Post -Body '{"user_id": "user_1", "amount": 40}' -ContentType "application/json"

transaction_id      balance
-----
fb197723-02c0-42d4-bbfc-fe2d8ec64d        60

PS C:\Users\user> Invoke-RestMethod -Uri "http://localhost:8080/process" -Method Post -Body '{"user_id": "user_2", "amount": 500}' -ContentType "application/json"

transaction_id      balance
-----
94b58262-1f93-412f-b1b2-eca3dfc5a724      500

```

GET method

```

+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands
PS C:\Users\user> Invoke-RestMethod -Uri "http://localhost:8080/accounts" -Method Get

ed_student user_1 user_2
-----
1          60      500

```

Scenario 1

```

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> python .\test_scripts\test_performance.py
Starting Test: 10 clients, 10000 reqs each...
Total Time: 1413.00s
Requests Per Second: 70.77
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

user_3 : 10000
user_1 : 10000
user_0 : 10000
user_4 : 10000
user_2 : 10000
user_8 : 10000
user_6 : 10000
user_5 : 10000
user_9 : 10000
user_7 : 10000

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/stats"

logging      counter
-----
5770,665229797363 5821,669531345367

```

Тестування продуктивності проводилося за допомогою асинхронного Python-скрипта, який симулював роботу 10 клієнтів, що одночасно виконували по 10 000 транзакцій кожен. У першому сценарії навантаження розподілялося на 10 різних акаунтів, що призвело до виконання 100 000 запитів за 1413 секунд із пропускнуою здатністю 70.77 запитів на секунду. Коректність було підтверджено фінальним балансом по 10 000 на кожному з десяти рахунків. Аналіз внутрішніх метрик показав, що внесок logging-service у загальний час склав приблизно 5770 секунд сумарно, а counter-service — 5821 секунду.

Scenario 2

```

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> python .\test_scripts\test_performance.py
Starting Test: 10 clients, 10000 reqs each...
Total Time: 1331.92s
Requests Per Second: 75.08
PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/accounts"

target_user
-----
100000

PS C:\Users\user\Documents\3 course\sem2\apz\hw\hw1> Invoke-RestMethod -Uri "http://localhost:8080/stats"

        logging        counter
        -----        -
5419,739134073257 5511,329410791397

```

Удругому сценарії, коли всі 10 клієнтів надсилали транзакції наодин спільний рахунок target_user, система продемонструвала вищу швидкість обробки, завершивши тест за 1331.92 секунди з показником 75.08 запитів на секунду. Таке підвищення продуктивності пояснюється меншими витратами ресурсів counter-service на оновлення одного і того ж ключа в хеш-таблиці порівняно з роботою з багатьма різними записами. Фінальний баланс рахунку target_user склав очікувані 100 000 одиниць, що підтверджує відсутність втрати даних під інтенсивним навантаженням . Сумарні показники часу для другого сценарію склали 5419.74 секунди для сервісу логування та 5511.33 секунди для сервісу лічильника.

Весь вихідний код проєкту збережений у репозиторії GitHub у гілці micro_basics.